
Technical Appendix — VITS, MAS, HiFi-GAN, and our training stack

Deep reference for the hard questions and the written report.
Everything here is grounded in the actual code under `src/vits_finetune/`. Read this if an examiner pushes past the slides.
Companion to `dl-presentation-brief.md`.

A. VITS — the full picture

A.1 What problem is VITS solving?

Classical neural TTS is **two stages**:

1. **Acoustic model** (Tacotron2, FastSpeech2): text → mel-spectrogram.
2. **Vocoder** (HiFi-GAN, WaveGlow): mel-spectrogram → waveform.

Two problems with two stages:

- **Train/inference mismatch (exposure bias)**. The vocoder is trained on *ground-truth* mels but at inference sees the acoustic model's *predicted* mels, which are blurrier — quality drops.
- **Mel is a lossy bottleneck**. Phase is discarded; fine detail is lost before the vocoder ever runs.

VITS removes the intermediate mel. It is a single model trained end-to-end from text to waveform, so there is no acoustic-model→vocoder seam to mismatch. The mel-spectrogram only ever appears as a *training loss target*, never as the thing the next module consumes.

A.2 VITS as a Conditional VAE

VITS frames TTS as a **conditional variational autoencoder (CVAE)** with a latent variable z (a frame-level acoustic representation, 192-dim here).

- **Posterior** $q(z | x_{\text{lin}})$ — the **posterior encoder** reads the *real audio's* linear spectrogram x_{lin} and produces z (mean + log- σ , sample). Only available in training (we have the audio then).
- **Prior** $p(z | \text{text})$ — the **text encoder** produces a prior over z from text. A **normalizing flow** f makes this prior more flexible: $z_p = f(z)$ lives in a space where a simple Gaussian prior is a good fit.
- **Decoder** $p(\text{waveform} | z)$ — the **HiFi-GAN-style decoder** turns z into the waveform.

The CVAE is trained to maximize the **Evidence Lower BOUND (ELBO)**:

$$\log p(x) \geq \mathbb{E}_q[\log p(x | z)] - \text{KL}(q(z | x_{\text{lin}}) \parallel p(z | \text{text}))$$

_____reconstruction_____/ _____prior/posterior match_____/

- The **reconstruction term** → our **mel L1 loss** (we reconstruct the waveform, then compare its mel to the target mel).
- The **KL term** → our **KL loss**, but with a twist: the prior must first be *aligned* to the audio frames (different lengths), which is what MAS does.

VITS then adds, on top of the ELBO:

- **Adversarial loss** (GAN) + **feature-matching loss** — for waveform realism, inherited from HiFi-GAN.
- **Duration loss** — to model rhythm at inference (no audio to align against).

So our total generator loss is:

$$L_G = 45 \cdot L_{\text{mel}} + 1 \cdot L_{\text{KL}} + L_{\text{dur}} + L_{\text{adv}}(G) + L_{\text{fm}}$$

$L_D = L_{\text{adv}}(D)$ # least-squares GAN, real vs. fake

(Weights from `config.TrainingConfig`: `mel_loss_weight=45`,
`kl_loss_weight=1`.)

A.3 The five components in detail

Component	HF submodule	What it computes	Train	Infer
Text encoder	<code>vits.text_encoder</code>	text → hidden states + prior	✓	✓

Component	HF submodule	What it computes	Train	Infer
		μ_θ , $\log \sigma_\theta$		
Posterior encoder	<code>vits.posterior_encoder</code>	linear spec $\rightarrow$ z , μ_ϕ , $\log \sigma_\phi$	✓	✗
Flow	<code>vits.flow</code>	invertible $z \rightarrow z_p$ (affine coupling)	✓	✓
Duration predictor	<code>vits.duration_predictor</code>	token $\rightarrow$ log-duration (stochastic)	✓	✓
Decoder	<code>vits.decoder</code>	$z \rightarrow$ waveform (transposed convs)	✓	✓
Discriminators	(ours) Discriminator	real/fake + feature maps	✓	✗

A.4 Normalizing flow — why it's there

A plain Gaussian prior $p(z|\text{text})$ is too simple to match the rich posterior $q(z|x)$. The flow f is a stack of **invertible** transforms (affine coupling layers). It maps $z \rightarrow z_p$ such that z_p is approximately Gaussian, so the KL between posterior and prior is computed in the well-behaved z_p space. At inference we sample $z_p \sim N$, run the flow **in reverse** (`reverse=True`) to get z , then decode. Invertibility is what makes “train forward, sample reverse” exact.

In code (`model.py`): `z_p = self.flow(z, spec_mask, reverse=False)` during training; inference uses HF's `vits(...)` which runs the flow in reverse internally.

A.5 Stochastic duration predictor

Speech is **one-to-many**: the same sentence can be said fast or slow, with different stress. A deterministic duration predictor (FastSpeech2-style) averages these out $\rightarrow$ flat prosody. VITS uses a **stochastic** duration predictor (a small flow-based model) trained via a variational lower bound on the log-likelihood of the MAS-derived durations. At inference it *samples* durations (`noise_scale_duration=0.8`), giving natural rhythm variation.

In code: `self.duration_predictor(..., durations=durations, reverse=False).mean()` returns the duration **loss** directly during training.

B. Monotonic Alignment Search (MAS) — step by step

The problem: we have `T_text` text tokens and `T_frames` audio frames, with `T_frames` $\gg$ `T_text`, and **no labels** saying which frames belong to which token. We need a hard alignment $A \in \{0,1\}^{(T_{\text{text}} \times T_{\text{frames}})}$ to (a) expand the prior to frame length and (b) supply duration targets.

The constraints: the alignment must be

- **monotonic** — token index never decreases as frames advance (speech is left-to-right);
- **surjective** — every frame maps to exactly one token, every token used;
- **non-skipping** — you can't jump over a token.

The objective: among all valid alignments, pick the one that maximizes the likelihood of the latent `z_p` under the aligned prior. This is a **dynamic-programming / Viterbi** search.

B.1 `neg_cross_entropy (alignment.py)`

Builds the per-(token, frame) log-likelihood matrix `Q[b, i, j]` = log-likelihood of frame `j`'s `z_p` under text token `i`'s Gaussian prior. Expanded for speed into four additive terms (constant, mean², z², cross) so it's all matmuls — no Python loop over tokens $\times$ frames. Shapes: prior `(B, 192, T_text)`, `z_p (B, 192, T_frames)` $\rightarrow$ `Q (B, T_text, T_frames)`.

B.2 `maximum_path (alignment.py)`

A vectorized DP, `@torch.no_grad` (it's a search, gradients flow through the *aligned prior* afterward, not through the `argmax`):

```
Forward (accumulate best score ending on token i at frame j):
    running[i] = Q[i,j] + max( running[i]          # stay on token i
                              , running[i-1] )    # advance from i-1
    direction[i,j] = (stay ? 1 : 0)                # remember the choice
Backward (traceback from last token, last frame):
    emit A[token, j] = 1 ; token -= (1 - direction[token, j])
```

The `reachable = text_index ≤ frame` mask enforces that token `i` can't start before frame `i` (you need at least one frame per earlier token). Padding is forced to “stay” so the traceback can't wander in batched, padded examples.

Output `A` is the hard 0/1 alignment. Then in `model.py`:

```
prior_mean      = prior_mean @ A          # expand prior to frame length
prior_log_stddev = prior_log_stddev @ A
durations       = A.sum(dim=2)             # how many frames per token → dur loss
```

Why MAS instead of attention? Attention-based aligners (Tacotron2) can skip/repeat/babble. MAS *guarantees* a monotonic, complete alignment by construction — far more stable, no alignment collapse.

C. The discriminator (HiFi-GAN) — `discriminator.py`

The decoder is a GAN generator; we built the **discriminator** from scratch. Two families, because audio has structure at multiple periods *and* multiple scales:

C.1 MPD — Multi-Period Discriminator

Speech is quasi-periodic (vocal-fold pitch). MPD reshapes the 1-D waveform of length `T` into a 2-D grid `(T/p, p)` for each **period** `p ∈ {2, 7, 13, 29, 37, 73, 97, 113, 137}` (primes, to avoid overlap), then runs 2-D convolutions down the `T/p` axis with kernel `(5,1)`. Each period sees a different slice of the harmonic structure. Channels: `1 → 32 → 128 → 512 → 1024 → 1024 → 1`.

`reshape_1D_to_2D` pads `T` up to a multiple of `p` first (`F.pad`).

C.2 MSD — Multi-Scale Discriminator

Captures structure at **3 time scales**: the raw audio, $\frac{1}{2}$ -rate, $\frac{1}{4}$ -rate (average-pooled with `AvgPool1d(4, 2)` between scales). Each scale is a stack of 1-D convs with large kernels `(15, 41, 41, 41, 5)` and grouped strides `(1, 4, 4, 4, 1)`. Catches longer-range envelope/formant structure MPD misses.

C.3 Outputs

Both return `(outputs, feature_maps)`. `Discriminator.forward` concatenates MPD + MSD branch outputs and *all* their feature maps. Outputs → adversarial loss; feature maps → feature-matching loss.

D. The losses — `losses.py` (exact formulas)

Loss	Formula (per branch, then summed)
Reconstruction	<code>mean(mel_pred - mel_tgt)</code> (L1)
KL	<code>mean_over_mask(logσ_p - logσ_q - 0.5 + 0.5·(z_p - μ_p)² / σ_p²)</code>
Discriminator (LSGAN)	<code>Σ mean((D(real) - 1)²) + mean(D(fake)²)</code>
Generator adversarial	<code>Σ mean((D(fake) - 1)²)</code>
Feature matching	<code>Σ_{branches} Σ_{layers} mean(f_{real} - f_{fake})</code>

Why LSGAN (least-squares) not vanilla GAN? Smoother gradients, no vanishing gradient when the discriminator is confident, much more stable — standard in HiFi-GAN/VITS.

Why feature matching? It tells the generator to match the *internal representations* the discriminator computes on real audio, not just fool its final verdict. Empirically the single biggest stabilizer for GAN vocoders.

Why mel-L1 weighted 45x? It's the primary supervision (spectral accuracy); the GAN terms are realism polish. 45 is the VITS paper's value.

Precision note: only **mel (45)** and **KL (1)** are configurable weights (`TrainingConfig.mel_loss_weight / kl_loss_weight`). Duration, adversarial, and feature-matching are added to `g_loss` **directly** — i.e. at an implicit weight of 1, with no config knob. Functionally fine; just not individually tunable.

E. The training step — `train.py`

The standard **alternating** GAN update, once per batch:

```
outputs = G.forward_train(batch)          # the whole VITS forward pass (App. A/B)
fake_wave = outputs['predicted_waveform']  # decoder output on a sliced segment
real_wave = outputs['target_waveform']     # matching real-audio segment

# (1) Discriminator step - learn to tell real from fake
```

```

d_outs_real, _ = D(real_wave)
d_outs_fake, _ = D(fake_wave.detach()) # .detach() → don't backprop into G
L_D = discriminator_loss(d_outs_real, d_outs_fake)
optD.zero_grad(); L_D.backward(); optD.step()

# (2) Generator step – fool D + match spectrum + ELBO + duration
fake_outs, fake_fmaps = D(fake_wave)
_, real_fmaps = D(real_wave)
L_G = 45·recon + 1·kl + dur + adv(fake_outs) + fm(real_fmaps, fake_fmaps)
optG.zero_grad(); L_G.backward(); optG.step()

```

- `.detach()` on the fake in the D step is critical — otherwise the discriminator’s loss would push gradients into the generator.
- Two **AdamW** optimizers, `lr=2e-4`.
- The code wraps these in Python **decorators** (`__model_step`, `__discriminator_step`, `__generator_step`, `__back_step_dec`) — a stylistic choice; functionally it’s the loop above.
- **Checkpointing**: every `checkpoint_every=1000` steps and at each epoch end, saving generator and discriminator separately (`*_G.pt` / `*_D.pt`), **resumable** via `--resume`.

E.1 Segment slicing (the speed trick)

The decoder is expensive, so we don’t decode the whole utterance. We crop a fixed `segment_size = 16384` samples (≈ 0.74 s at 22.05 kHz, = 64 mel frames at hop 256) at a random offset from `z`, decode just that, and slice the matching real-waveform window as the target. This is exactly HiFi-GAN/VITS practice — the decoder is fully convolutional so a short crop trains it fine, and memory/time drop by an order of magnitude.

F. Audio front-end — `audio.py` / `config`

Param	Value	Meaning
<code>sampling_rate</code>	22050	matches vits-ljs
<code>n_fft</code>	1024	STFT window
<code>hop_length</code>	256	frame stride (→ ~86 frames/s)
<code>win_length</code>	1024	STFT window length
<code>n_mels</code>	80	mel bands
<code>spectrogram_bins</code>	513	<code>n_fft/2 + 1</code> , posterior-encoder input

Two spectrograms per clip: a **linear** spectrogram (posterior-encoder input) and an **80-band mel** (reconstruction-loss target).

G. Inference path (how before/after is produced)

At inference there is no posterior encoder. `synthesize.py` calls HF's `model.vits(input_ids=...)` which internally: text-encodes → predicts durations (stochastic, sampled) → expands the prior → samples $z_p \sim N$, runs the flow in reverse → decodes to waveform. The **only** switch between the pretrained and the fine-tuned model is which weights are loaded (`--checkpoint`), so the before/after comparison is on identical text and identical inference code.

Noise knobs: `noise_scale=0.667` (latent), `noise_scale_duration=0.8` (rhythm). Lower = more deterministic/flat, higher = more varied.

H. The dataset pipeline — exact stages

Stage	Script	In → Out
1. Synthesize	<code>data/bake_dataset.py</code>	LJSpeech text → clean VITS wavs (resumable, error-logged)
2. Voice-convert	B1 RVC (Docker)	clean wavs → B1-droid wavs (48 kHz)
3. Resample + meta	<code>data/prepare_b1_dataset.py</code>	48 kHz → 22.05 kHz PCM16 + <code>metadata.csv</code> (HF AudioFolder, <code>ProcessPoolExecutor</code> , <code>soxr</code>)
4. Publish	<code>data/push_b1_dataset.py</code>	folder → <code>Dmitri13/ljspeech-b1</code> on HF Hub

Dataset card: 13,100 examples, 22050 Hz, {audio, text}, public, ~24 h. `text` = original LJSpeech *normalized* transcript (RVC changes timbre, not words). Train/eval split: `train[:95%]` / `train[95%:]` (config).

RVC = Retrieval-based Voice Conversion — converts the *timbre* of a source voice to a target voice while keeping the linguistic content and prosody. We ran the “B1 Battle Droid” RVC model in a Docker container against the baked wavs.

H.1 RVC step — exact setup (docker /)

- **Container:** CUDA 12.8 image cloning RVC-Project, with `hubert_base.pt` pre-baked in. Torch pinned to `2.11.0+cu128`; `fairseq` stripped from RVC's requirements and reinstalled against `omegaconf==2.0.6` (dependency-hell fix).

- **B1 voice model:** Homiebear/B1BattleDroid — a `.pth` checkpoint + FAISS `.index` (the “retrieval” half of RVC).
- **Inference params** (`docker/run.py`): `index_rate=0.75` (75% target-voice characteristics vs. 25% source), `protect=0.33` (preserve consonant clarity), `f0_up_key=0` (no pitch shift), `f0_method = pm` (fast) or `rmvpe` (accurate).
- **Scale features:** batch watch-mode + sharding for multi-GPU; per-file error log. A `diag.py` profiler was used to debug a CUDA 12.8-vs-driver mismatch.

H.2 The OTHER “droid” path — DSP effects (NOT the dataset)

The repo also contains a **DSP-based** droid approach — don’t confuse it with RVC: - `effects.py` — `apply_b1_droid()` is a 6-stage filter chain: 40 Hz ring modulation, +3-semitone pitch shift, 250–6000 Hz band-pass, 2.5 kHz presence boost, `tanh` soft-clip, comb-filter (5 ms / 0.4 feedback). It produced the fun `demo_*.wav` clips (normal / accent / dying / excited / stutter). - `augmentation/magic.py` — an **earlier** (“Variant A”) augmentation that applied that DSP chain + a phoneme-level Russian accent to raw LJSpeech audio. - **These are explicitly NOT used for training or evaluation.** The shipped dataset `Dmi1tr13/ljspeech-b1` is the **RVC** path (Variant B) above. The DSP effects are demo/garnish; a fixed filter wouldn’t test whether the *model* learned a voice, which is the whole point of fine-tuning.

I. Evaluation internals — `evaluation/`

- `_transcribe` — Whisper `base`, `language='en'`, lower-cased.
- `compute_wer` / `compute_cer` — `jiwer.wer/cer(references, hypotheses)`.
- `compute_mcd` — `pymcd Calculate_MCD(MCD_mode='dtw')` between reference and generated wav (DTW handles length differences).
- `evaluate()` — reads the manifest (`{id, text, ref_audio}`), finds each `<id>.wav`, computes MCD per clip (NaN-tolerant mean), then corpus WER/CER. `--mock` skips Whisper (MCD only) for offline wiring tests.
- `compare()` — prints the before/after table from a list of `EvalResult`.

Metric intuition:

- **WER** = (substitutions + insertions + deletions) / reference words. **CER** = same at character level (catches small mispronunciations WER rounds to a whole word).
 - **MCD (dB)** = $(10/\ln 10) \cdot \sqrt{(2 \cdot \sum (c_{\text{gen}} - c_{\text{ref}})^2)}$ over mel-cepstral coefficients, DTW-aligned. ~5–8 dB $\approx$ decent TTS; 0 = identical.
-

J. Known limitations & environment (honest)

- **Mel length mismatch (~3 frames)**. STFT uses `center=False` (`audio.py:27`), so a 16384-sample decoder segment $\rightarrow$ 61 mel frames, while the sliced target is 64 frames. `forward_train` takes `min(...)` of the two, so it trains fine but truncates ~3 frames (~5%) of the reconstruction target. Clean fix: `center=True` or pad the predicted waveform to a 64-frame multiple.
- **Duration loss unweighted** (see §D precision note) — no config knob.
- **No real fine-tuned metrics yet** — the full GPU run is pending; the only measured number is the pretrained baseline. `models/finetuned/training_log.json` is a placeholder mock (still references the old SpeechT5 plan).
- **Tokenizer**: the model **already runs phonemes** — `vits-ljs`'s `VitsTokenizer` uses `espeak-ng` (`phonemize=True`), so training/inference are phoneme-based (needs the `phonemizer` pkg + `espeak-ng` system binary). The repo's `tokenization/tokenizer.py` (`to_characters / to_phonemes` as identity stubs) is a separate *char-vs-phoneme ablation*, future work — not the live path.
- **Environment grit**: Torch 2.11.0+cu128, Transformers 5.9.0, Pydantic 2.13.4 are pinned exactly; **espeak-ng** is a system dependency (not pip). The RVC container needed careful CUDA/ `fairseq` / `omegaconf` pinning.
- **Decorator-based training loop** (`train.py`) — the epoch/batch loop lives in a decorator (`__back_step_dec`) wrapping `stepof5G0ATS(self, batch)`, which is then called with no args. Works, but the signature is misleading; document it if you maintain it.

K. Glossary (one-liners)

- **VITS** — Variational Inference with adversarial learning for end-to-end TTS.

- **CVAE** — Conditional Variational AutoEncoder; latent-variable generative model.
- **ELBO** — Evidence Lower BOund; the VAE training objective.
- **KL divergence** — distance between two distributions; ties prior to posterior.
- **Normalizing flow** — stack of invertible maps; turns a complex distribution into a simple (Gaussian) one and back.
- **MAS** — Monotonic Alignment Search; DP search for the text $\rightarrow$ frame alignment.
- **HiFi-GAN** — GAN vocoder; VITS borrows its decoder + discriminators.
- **MPD / MSD** — Multi-Period / Multi-Scale Discriminator.
- **LSGAN** — least-squares GAN loss; stable adversarial training.
- **Feature matching** — L1 between discriminator activations on real vs. fake.
- **Mel-spectrogram** — perceptually-scaled spectrogram; here a *loss target*.
- **RVC** — Retrieval-based Voice Conversion; changes timbre, keeps content.
- **WER / CER** — Word / Character Error Rate (intelligibility, lower better).
- **MCD** — Mel Cepstral Distortion (spectral fidelity, dB, lower better).
- **Exposure bias** — train/inference mismatch in two-stage models VITS avoids.
- **Segment slicing** — train the decoder on a short random audio crop.

L. References

1. Kim, Kong & Son, “**Conditional VAE with Adversarial Learning for End-to-End TTS (VITS)**”, ICML 2021. arXiv:2106.06103 — *the paper*.
2. Kong, Kim & Bae, “**HiFi-GAN**”, NeurIPS 2020. arXiv:2010.05646 — decoder + MPD/MSD discriminators.
3. Ren et al., “**FastSpeech 2**”, ICLR 2021. arXiv:2006.04558 — Option A.
4. Shen et al., “**Tacotron 2**”, ICASSP 2018. arXiv:1712.05884 — Option B.
5. Ito, “**The LJ Speech Dataset**”, 2017. keithito.com/LJ-Speech-Dataset.
6. Radford et al., “**Whisper**”, 2022 — ASR for WER/CER.
7. HF `kakao-enterprise/vits-ljs` — our pretrained checkpoint.
8. Our dataset — 🤗 `Dmitr13/ljspeech-b1` .